

LAMMPS pip-wheel build — complete reproduction reference

Part A: Docker / manylinux_2_28 wheel creation

Part B: Native apt build (Ubuntu / Pop!_OS)

Part C: Target machine runtime requirements

Contents

I	Part A — Portable manylinux_2_28 wheel (Docker)	3
1	Pull the image and start the container	3
2	What is already in the image (no dnf action needed)	3
3	dnf packages to install inside the container	4
3.1	Core build dependencies (defensive install, most already in image)	4
3.2	EPEL repository (needed for voro++ and EPEL-only packages)	5
3.3	ffmpeg (RPM Fusion — not in base repos or EPEL on AlmaLinux 8)	5
3.4	Cython (for uv-managed Python, required by ML-IAP mliappy path)	5
4	Two Python interpreters — why both are needed	5
5	cmake configure — every error hit, in order, with fixes	7
5.1	Error 1: Python not found	7
5.2	Error 2 (transient): cmake not editing Python_EXECUTABLE	7
5.3	Non-blocking warnings (left as-is)	7
5.4	Error 3: dependency-not-found for downloadable packages	7
5.5	Error 4: stale cache — ADIOS2	8
5.6	FFMPEG: not in default repos, needs RPM Fusion	9
6	Final cmake configure command	9
7	Build	11
8	Build the wheel	11
9	auditwheel: inspect, then repair	12
9.1	Install auditwheel and inspect first	12
9.2	First repair attempt — and the two runtime failures it caused	12
9.3	Fix: exclude MPI and curl/ldap from vendoring	13
9.4	What remains bundled	14

10 Verification on target machine (Pop!_OS)	14
10.1 Installed-packages check	14
10.2 Multi-rank MPI test	14
10.3 NetCDF dump (exercises HDF5/krb5/gssapi chain)	15
10.4 Embedded Python package (PKG_PYTHON)	15
10.5 KIM API presence	16
11 Complete command sequence, start to finish	16
 II Part B — Native apt build (Ubuntu / Pop!_OS)	 19
12 Base build tooling	19
13 CMake + cmake (Kitware repo)	19
14 Compilers and MPI (OpenMPI)	20
15 Python and wheel tooling	20
16 All libraries (MLIAP, EXTRA-PAIR, PYTHON, COLVARS, FFT, dump stack)	20
17 cmake configure for native build	21
18 Post-build sanity checks	22
19 Build the wheel (native)	23
20 Runtime smoke test (native build)	23
21 Portability caveats for the native-built wheel	24
 III Part C — Target machine runtime requirements	 25
22 Debian / Ubuntu / Pop!_OS target	25
23 RHEL / AlmaLinux / Rocky Linux target	25
24 Install the wheel	26
25 KIM model data (either target)	26
26 Quick functional verification on target	26
27 Recommended: pin this wheel via a uv project	27
A Unrelated: A100 CUDA export fix	28

Part I

Part A — Portable manylinux_2_28 wheel (Docker)

Scope

Full, step-by-step reproduction record sufficient to recreate the portable `lammps` wheel from a clean Docker pull. Covers: image selection, *every* `dnf` package installed inside the container (and why), Python interpreter setup, `cmake` configure with every error encountered in order and its fix, build, wheel packaging via `python/install.py`, `auditwheel` repair (including the two runtime failures the first repair attempt caused and how they were resolved), and functional verification on a separate Pop!_OS target.

Source image: `quay.io/pypa/manylinux_2_28_x86_64:latest` (stock, unmodified, pulled 2026-08-08). Nothing was pre-baked; all packages were added via `dnf` inside a running container or fetched by `cmake`'s own `DOWNLOAD_*` mechanism.

LAMMPS version built and verified: 2026.2.11.

1 Pull the image and start the container

Run from the directory that already contains the LAMMPS source tree (`cmake/`, `src/`, `python/`, etc.).

```
docker pull quay.io/pypa/manylinux_2_28_x86_64:latest

docker run -it --rm \
  -v "$PWD":/work \
  -w /work \
  quay.io/pypa/manylinux_2_28_x86_64:latest
```

`/work` is bind-mounted from the host. Anything written *outside* `/work` (e.g. `/root/...`) is **lost on container restart** — this matters for the uv-managed Python installed in Section 4.

2 What is already in the image (no `dnf` action needed)

The following were confirmed present in the image pull used for this build — verified by `cmake`'s own “Found” lines during configure, not assumed:

- **Compiler:** `gcc-toolset-14` (`gcc/g++/gfortran 14.2.1`) at `/opt/rh/gcc-toolset-14/root/usr/bin/`
- **cmake:** pip-installed `cmake 4.4` (on `PATH`, supersedes the system's `cmake 3.26.5`); system `cmake 3.26.5` *also* installed at `/usr/bin/cmake`
- **ccmake:** NOT in the pip-installed `cmake` wheel (no curses); **IS** present at `/usr/bin/ccmake` via the system `cmake` RPM. *Verify with:* `which -a cmake ccmake`

- **git** (required by cmake DOWNLOAD_* for source fetches)
- **MPI**: OpenMPI 3.1 headers and libs, **mpicxx** wrapper functional, found at `/usr/lib64/openmpi/`
- **OpenMP** 4.5
- **ZLIB** 1.2.11
- **FFTW3** 3.3.5 (threaded, found via pkg-config)
- **JPEG** (libjpeg-turbo 62), **PNG** (libpng 1.6.34)
- **CURL** 7.61.1 (libcurl-devel headers present)
- **NetCDF**, **LAPACK**/**OpenBLAS**, **GSL**, **HDF5**, **SWIG**, **BLAS**, **PkgConfig**, **Threads**
- **manylinux-blessed static Python 3.12** at `/opt/python/cp312-cp312/bin/python`

These are image-state dependent. For a reproducible future build, install the dnf packages in Section 3 regardless — they are no-ops if already satisfied, and will fail loudly if not.

3 dnf packages to install inside the container

Install these in order. The groupings explain *why* each is needed.

3.1 Core build dependencies (defensive install, most already in image)

```
dnf install -y \
    openmpi openmpi-devel \
    fftw-devel \
    libjpeg-turbo-devel \
    libpng-devel \
    zlib-devel \
    libcurl-devel \
    netcdf-devel \
    openblas-devel \
    lapack-devel \
    ncurses-devel \
    git
```

Notes:

- **openmpi-devel** — headers and **mpicxx** wrapper; MPI was confirmed present in the image, but installing explicitly guarantees it across image updates.
- **ncurses-devel** — defensive for **ccmake**; not actually needed if `/usr/bin/ccmake` is already present.
- The rest map to libraries confirmed linked into `liblammps.so` by `auditwheel show`.

After installing `openmpi-devel`, confirm the **mpicxx** wrapper is on PATH:

```
which mpicxx || export PATH=/usr/lib64/openmpi/bin:$PATH
```

3.2 EPEL repository (needed for voro++ and EPEL-only packages)

```
dnf install -y epel-release
dnf update -y
```

3.3 ffmpeg (RPM Fusion — not in base repos or EPEL on AlmaLinux 8)

```
dnf install -y \
    https://download1.rpmfusion.org/free/el/rpmfusion-free-release-8.noarch.rpm

dnf install -y ffmpeg
```

ffmpeg is never compiled against or linked into liblammps.so — it is invoked by LAMMPS as an external subprocess at runtime for `dump movie`. It has no effect on wheel portability (`auditwheel` never touches it). Not installing it here means `cmake` silently leaves the `dump movie` feature off; installing it enables the feature.

Two-step fix required: after `dnf install -y ffmpeg`, `cmake` detects the binary but leaves `WITH_FFMPEG` off by default. Pass `-D WITH_FFMPEG=yes` explicitly in the `configure` command (Section 6).

3.4 Cython (for uv-managed Python, required by ML-IAP mliappy path)

This cannot be done until the uv Python is set up (Section 4). See that section for the correct invocation.

4 Two Python interpreters — why both are needed

This is the single most important structural fact about the manylinux wheel build. Two separate Python operations need Python for different reasons:

- `$PYBIN = /opt/python/cp312-cp312/bin` — the manylinux-blessed CPython. Statically linked (`Py_ENABLE_SHARED=0`), **no** `libpython3.12.so` anywhere under its prefix (confirmed via `find /opt/python/cp312-cp312 -iname "libpython*" returning nothing`). Correct for stamping the wheel's ABI tag (`cp312-cp312-manylinux_2_28_x86_64`). **Cannot** satisfy `PKG_PYTHON=yes` which needs `Development.Embed` — an actual shared `libpython` to link against.
- `$UVPY` — a uv-managed CPython (`python-build-standalone`). Ships `libpython3.12.so.1.0` (shared, `Py_ENABLE_SHARED=1`). Used *only* to satisfy the `cmake configure/compile` step for `PKG_PYTHON`.

Two-pass split: compile `liblammps.so` using `$UVPY` (needs the shared lib), then build the actual wheel using `$PYBIN` (needs the manylinux-correct ABI tag). Once `liblammps.so` is on disk, it carries no memory of which interpreter built it; the wheel-build step can use a different one.

Caveat: `uv`'s installed Pythons live under `/root/.local/share/uv/...`, not under `/work`. This directory is **lost on container restart** — happened once during this build. `$UVPY` must be reinstalled and re-exported before continuing after any restart.

Set up both interpreters

```
Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env
uv --version

# Install embedding-capable Python 3.12 via uv
uv python install 3.12
UVPY=$(uv python find 3.12)
echo $UVPY

# Verify it has the shared lib (must print 1)
$UVPY -c "import sysconfig; print(sysconfig.get_config_var('Py_ENABLE_SHARED'))"
find $(dirname $(dirname $UVPY)) -iname "libpython3.12*"

# Set the manylinux python (for wheel stamping and auditwheel)
export PYBIN=/opt/python/cp312-cp312/bin
$PYBIN/python --version
```

Install Cython under uv Python (required for ML-IAP `mliappy`)

`PKG_ML-IAP`'s Python model path (`mliappy`) needs a working `cythonize`. Plain `$UVPY -m pip install cython` was tried first and rejected with `externally-managed-environment` (PEP 668, since `uv` manages this interpreter). The correct invocation is:

```
uv pip install --python $UVPY cython

# Confirm cythonize is discoverable
find $(dirname $(dirname $UVPY)) -name "cythonize"
```

5 cmake configure — every error hit, in order, with fixes

5.1 Error 1: Python not found

```
CMake Error: Could NOT find Python
(missing: Python_INCLUDE_DIRS Python_LIBRARIES
Interpreter Development Development.Module Development.Embed)
```

Python_EXECUTABLE was not specified on the first attempt, so FindPython searched default paths and found nothing usable.

Fix: always pass `-D Python_EXECUTABLE=$UVPY` explicitly on every cmake invocation. See Section 4 for why `$UVPY` and not `$PYBIN`.

5.2 Error 2 (transient): ccmake not editing Python_EXECUTABLE

Attempting to change PYTHON_EXECUTABLE (old, deprecated FindPythonInterp name) inside ccmake silently failed. Modern CMake (3.12+ style FindPython) uses Python_EXECUTABLE — capital P.

Fix: do not fight through ccmake; wipe build/ and pass `-D Python_EXECUTABLE=...` on the initial cmake invocation. Changing the interpreter after first configure does not reliably re-derive the cached Python_LIBRARY/Python_INCLUDE_DIR, risking a mismatched pairing that compiles but breaks at import.

5.3 Non-blocking warnings (left as-is)

These appeared during configure and did *not* stop the build — they only disable specific optional features:

- Could NOT find PNetCDF — parallel NetCDF variant, optional; core NetCDF still found and used.
- Could NOT find TBB_MALLOC — only affects Intel-package performance tuning (dpd/intel), not correctness.
- HDF5 C compiler wrapper unable to compile minimal program — warning only; HDF5 was still found and linked.

5.4 Error 3: dependency-not-found for downloadable packages

Each failed with a find_package error when the system package was absent. LAMMPS's cmake supports fetching and building each from source in-tree via a DOWNLOAD_* flag. Decision made mid-session: use DOWNLOAD_*=yes for *all* of these for reproducibility — every dependency gets the same low-glibc-baseline compile as the rest of the build, which is more portable than whatever a distro repo would have shipped.

```
Could NOT find VORO
n2p2 not found ... set DOWNLOAD_N2P2=ON
MDI download requested (no system package exists for MDI)
Could NOT find KIM-API
```

```
PLUMED / QUIP not found
Eigen3 not found
```

Fix: pass all of the following in the cmake invocation:

```
-D DOWNLOAD_VORO=yes
-D DOWNLOAD_N2P2=yes
-D DOWNLOAD_MDI=yes
-D DOWNLOAD_KIM=yes
-D DOWNLOAD_PLUMED=yes
-D DOWNLOAD_QUIP=yes
-D DOWNLOAD_EIGEN3=yes
```

Each `DOWNLOAD_X=yes` only takes effect if the corresponding `PKG_X=yes` (or the package-specific enable flag, e.g. `PKG_ML-HDNNP` for `n2p2`, `PKG_ML-QUIP` for `QUIP`) is also set. A `DOWNLOAD` flag alone with the package disabled does nothing.

Note: `vorop++/vorop++-devel` and `eigen3-devel` *are* available as real `dnf/apt` packages (via EPEL and default repos, respectively) and can be used as a system install if preferred. The `DOWNLOAD_*=yes` route was chosen here for a uniform low-glibc baseline.

```
System-package alternative (not used in this build, recorded for reference):
dnf install -y epel-release
dnf install -y vorop++ vorop++-devel
# then use: -D PKG_VORONOI=yes -D DOWNLOAD_VORO=no

# Eigen3 system package alternative:
dnf install -y eigen3-devel
# then omit DOWNLOAD_EIGEN3=yes
```

5.5 Error 4: stale cache — ADIOS2

```
CMake Error: By not providing "FindADIOS2.cmake" ...
Could not find a package configuration file provided by "ADIOS2"
```

`PKG_ADIOS` was `ON` in `CMakeCache.txt` despite not being in the active flag list — leftover from an earlier attempt, since `build/` was not wiped before that particular reconfigure. No `DOWNLOAD_ADIOS2` flag exists in LAMMPS's cmake (unlike the packages above) — ADIOS2 must be a real system install if wanted.

Fix used: disabled it, since it was not actually needed:

```
cmake -B build -D PKG_ADIOS=no
```

To avoid this entirely: always wipe build/ before a fresh configure (see Section 6).

5.6 FFMPEG: not in default repos, needs RPM Fusion

```
Could NOT find FFMPEG (missing: FFMPEG_EXECUTABLE)
```

```
Plain dnf fails:
dnf install -y ffmpeg
# => No match for argument: ffmpeg

# Add RPM Fusion, then install:
dnf install -y \
    https://download1.rpmfusion.org/free/el/rpmfusion-free-release-8.noarch.rpm
dnf install -y ffmpeg
```

After installing ffmpeg and reconfiguring, the binary is detected but the cmake feature variable remains off:

```
FFMPEG_EXECUTABLE:FILEPATH=/usr/bin/ffmpeg
WITH_FFMPEG:BOOL=OFF
```

Finding the binary and enabling the feature are two separate cmake variables. Pass `-D WITH_FFMPEG=yes` explicitly in the configure command.

6 Final cmake configure command

```
Start from /work (the LAMMPS source root)
cd /work
rm -rf build && mkdir build && cd build

cmake -B . -S ../cmake \
    -D BUILD_SHARED_LIBS=yes \
    -D CMAKE_BUILD_TYPE=Release \
    -D PKG_MLIAP=yes \
    -D PKG_EXTRA-PAIR=yes \
    -D PKG_KIM=yes \
    -D PKG_MDI=yes \
    -D PKG_PYTHON=yes \
    -D PKG_COLVARS=yes \
    -D PKG_NETCDF=yes \
    -D PKG_VORONOI=yes \
    -D PKG_ML-HDNNP=yes \
    -D PKG_PLUMED=yes \
    -D PKG_ML-QUIP=yes \
    -D PKG_ADIOS=no \
```

```

-D MLIAP_ENABLE_PYTHON=no \
-D BUILD_MPI=yes \
-D DOWNLOAD_KIM=yes \
-D DOWNLOAD_MDI=yes \
-D DOWNLOAD_N2P2=yes \
-D DOWNLOAD_VORO=yes \
-D DOWNLOAD_PLUMED=yes \
-D DOWNLOAD_QUIP=yes \
-D DOWNLOAD_EIGEN3=yes \
-D WITH_FFMPEG=yes \
-D Python_EXECUTABLE=$UVPY

```

Key flag notes:

- `MLIAP_ENABLE_PYTHON=no` is deliberate — this project needs `PKG_MLIAP` (C++/ML potentials) and `PKG_PYTHON` (importable from `lammps import lammps` module), but *not* Python-callable ML potentials at simulation runtime, which is a separate, narrower feature than either.
- `Python_EXECUTABLE=$UVPY` (not `$PYBIN`) — uv Python has `libpython3.12.so.1.0`; the manylinux python does not. `PKG_PYTHON=yes` requires it.
- `PKG_ADIOS=no` — ADIOS2 has no cmake `DOWNLOAD` flag and was not needed; this explicitly disables it even if it was cached from an earlier attempt.

Expected tail output:

```

Configuring done (X.Xs)
Generating done (X.Xs)
Build files have been written to: /work/build

```

With no `Configuring incomplete, errors occurred!` line. The full enabled-packages summary (including KIM, MDI, PYTHON, COLVARS, NETCDF, VORONOI, ML-HDNNP, PLUMED, ML-QUIP, ML-IAP, ML-PACE, etc.) and compiler/MPI/FFT settings block should print above that.

The actual enabled packages list from this build was: AMOEBA; APIP; ASPHERE; BOCS; BODY; BPM; BROWNIAN; CG-DNA; CG-SPICA; CLASS2; COLLOID; COLVARS; COMPRESS; CORESHELL; DIELECTRIC; DIFFRACTION; DIPOLE; DPD-BASIC; DPD-MESO; DPD-REACT; DPD-SMOOTH; DRUDE; EFF; ELECTRODE; EXTRA-COMMAND; EXTRA-COMPUTE; EXTRA-DUMP; EXTRA-FIX; EXTRA-MOLECULE; EXTRA-PAIR; FEP; GRANULAR; GRAPHICS; H5MD; INTEL; INTERLAYER; KIM; KSPACE; LATBOLTZ; LEPTON; MACHDYN; MANIFOLD; MANYBODY; MC; MDI; MEAM; MESONT; MGPT; MISC; ML-HDNNP; ML-IAP; ML-PACE; ML-POD; ML-QUIP; ML-RANN; ML-SNAP; ML-UF3; MOFFF; MOLECULE; MOLFILE; NETCDF; OPENMP; OPT; ORIENT; PERI; PHONON; PLUGIN; PLUMED; PTM; PYTHON; QEQ; QMMM; QTB; REACTION; REAXFF; REPLICA; RHEO; RIGID; SHOCK; SMTBQ; SPH; SPIN; SRD; TALLY; UEF; VORONOI; YAFF

7 Build

```
make -j$(nproc)
```

This is a long build — KIM, N2P2, PLUMED, QUIP, PACE, MDI, Voro++, and Eigen3 all compile from source in-tree. Expect significant time.

Expected final lines:

```
[100%] Built target lammps
[100%] Built target lmp
```

A `pylammps/_pylammps.so` SWIG-generated target may also build (triggered by `PKG_PYTHON` + SWIG being present in the image). This is a separate, older binding mechanism, unrelated to the `python/lammps ctypes` package this document packages. Ignore it; it is not included in the wheel.

8 Build the wheel

`python/` already contains LAMMPS's own `install.py` and `setup.py` — no custom packaging script is needed. Switch to `$PYBIN` here (not `$UVPY`), so the wheel's ABI tag is manylinux-correct:

```
cd /work/python

$PYBIN/python install.py \
  -p lammps \
  -l ../build/liblammps.so \
  -v ../src/version.h \
  -n \
  -w .
```

Flag meanings:

- `-p lammps` — package directory name
- `-l ../build/liblammps.so` — path to the compiled shared lib
- `-v ../src/version.h` — LAMMPS version source file
- `-n` — build wheel only (do not install into the current env)
- `-w .` — output directory for the `.whl` file

`install.py` copies the `python/` tree to a scratch directory, copies `liblammps.so` into the `lammps/` package folder (so `setup.py`'s `package_data` picks it up), builds a venv from the invoking interpreter, and runs the wheel build inside that venv — calling it via `$PYBIN/python` is not cosmetic, it controls the wheel's ABI tag.

Output at this stage: `lammps-2026.2.11-cp312-cp312-linux_x86_64.whl` — correctly Python-tagged, but *not yet* portable (glibc symbol versions unconstrained, `linux_x86_64` generic tag).

9 auditwheel: inspect, then repair

9.1 Install auditwheel and inspect first

```
$PYBIN/pip install auditwheel patchelf

$PYBIN/python -m auditwheel show \
    lammps-2026.2.11-cp312-cp312-linux_x86_64.whl
```

Confirmed clean: no blacklisted symbols, resolves to `manylinux_2_27_x86_64`.

Note: `auditwheel show` does not list MPI libraries because it does not intend to vendor them by default policy. Confirm MPI is actually linked:

```
ldd ../build/liblammps.so | grep -i mpi
```

MPI was confirmed linked even though `auditwheel show` did not list it — the repair step below handles this.

9.2 First repair attempt — and the two runtime failures it caused

```
DO NOT USE THIS FORM --- documents the failure, not the fix
$PYBIN/python -m auditwheel repair \
    lammps-2026.2.11-cp312-cp312-linux_x86_64.whl \
    -w wheelhouse/
```

This repair, without any `-exclude` flags, vendored: MPI (`libmpi.so.40`, `libmpi_cxx.so.40`, and their transitive dependents), `libcurl.so.4`, `libldap-2.5.so.0`, `libkrb5`, and related libs (KIM's `kim query` pulls in `curl`, which transitively pulls `ldap/krb5`).

It also correctly stripped an explicit `libpython3.12.so.1.0` dependency from `liblammps.so` with the warning:

```
WARNING: Removing libpython3.12.so.1.0 dependency from lammps/liblammps.so.
Linking with libpython is forbidden for manylinux/musllinux wheels.
```

This is expected and correct.

Installed into a clean venv *on the container itself* — worked, `import lammps` returned ok.

Installed on the actual Pop!_OS target via `uv pip install` — **two separate failures:**

Failure 1 — MPI crash at import:

```
-----  
Sorry! You were supposed to get help about:  
  opal_init:startup:internal-failure  
But I couldn't open the help file:  
  /usr/lib64/openmpi/share/openmpi/help-opal-runtime.txt:  
  No such file or directory.  
-----  
*** An error occurred in MPI_Init  
*** on a NULL communicator
```

Cause: the bundled MPI libraries were the RHEL/AlmaLinux build, with internal data paths hardcoded to `/usr/lib64/openmpi/share/...` — a path layout that does not exist on Debian-family Pop!_OS. Pop!_OS already had its own working OpenMPI 4.1.2 installed, entirely unused because the bundled copy's RPATH took priority.

Failure 2 — segfault at interpreter shutdown (after a clean run):

```
Segmentation fault (11)  
...  
/usr/lib/x86_64-linux-gnu/libc.so.6(free+0x1e)  
/usr/lib/x86_64-linux-gnu/libldap-2.5.so.0(+0x1007b)
```

Cause: the bundled libldap and the system's own separately-loaded libldap-2.5.so.0 (pulled in via NSS at process exit) both get unloaded at teardown. A version/ABI mismatch triggers a double-free inside libc's `free()`. Same failure class as MPI — a bundled library colliding with a system-loaded library of the same effective identity at a point `auditwheel`'s static analysis cannot see (runtime `dlopen`/NSS-plugin behavior, not a direct `DT_NEEDED` entry).

9.3 Fix: exclude MPI and curl/ldap from vendoring

```
cd /work/python/wheelhouse  
  
$PYBIN/python -m auditwheel repair \  
  ../lammps-2026.2.11-cp312-cp312-linux_x86_64.whl \  
  -w . \  
  --exclude libmpi.so.40 \  
  --exclude libmpi_cxx.so.40 \  
  --exclude libldap-2.5.so.0 \  
  --exclude libcurl.so.4
```

Excluding `libmpi.so.40/libmpi_cxx.so.40` automatically dropped their transitive dependents (`libopen-rte.so.40`, `libopen-pal.so.40`, `libopen-orted-mpir.so`) without needing to list them separately. This produces a wheel that *requires* the target system to have its own compatible OpenMPI and curl/ldap installed — the correct contract for an MPI-linked wheel.

Result:

```
Fixed-up wheel written to:
lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
```

Note: auditwheel found the wheel was eligible for the higher-priority `manylinux_2_27` tag and used both tags.

9.4 What remains bundled

After the fix, `lammps.libs/` on the target contained: FFTW3(+omp), OpenBLAS, GSL/GSLCBLAS, HDF5(+HL), NetCDF, libkim-api, libjpeg, libpng16, libzstd, libgfortran, libquadmath, libgomp, plus `libgssapi_krb5/libkrb5/libselinux/libtirpc/libcrypto` (transitive, possibly from HDF5/NetCDF auth features rather than curl, since curl itself was excluded). These were not excluded because the functional tests below exercised the codepaths that would trigger them (netCDF dump, KIM) and passed cleanly with exit code 0.

10 Verification on target machine (Pop!_OS)

Copy the wheel file out of the container (or from the bind-mounted `/work` directory) to the target machine, then:

```
uv pip install --force-reinstall \
    lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
```

`-force-reinstall` is required when re-testing a rebuilt wheel with an unchanged version string, or `uv/pip` will skip the reinstall.

10.1 Installed-packages check

```
python -c "from lammps import lammps; l = lammps(); print(l.installed_packages)"
echo $?
```

Expected: full compiled-in package list printed, exit code 0. Confirmed output included KIM, MDI, PYTHON, COLVARS, NETCDF, VORONOI, ML-HDNNP, PLUMED, ML-QUIP, ML-IAP, ML-PACE.

10.2 Multi-rank MPI test

```
mpirun -np 2 python -c \
    "from lammps import lammps; l = lammps(); \
    l.commands_string('units lj\natom_style atomic\n\
    lattice fcc 0.8442\nregion box block 0 4 0 4 0 4\n\
    create_box 1 box\ncreate_atoms 1 box\nmass 1 1.0\n\
```

```
pair_style lj/cut 2.5\npair_coeff 1 1 1.0 1.0 2.5\nrun 10'); \
    print('mpi ok')" -quiet
echo "exit: $?"
```

Passed on both ranks with identical energy/pressure output to a single-rank run. Exit code 0.

10.3 NetCDF dump (exercises HDF5/krb5/gssapi chain)

```
python -c "
from lammps import lammps
l = lammps()
l.commands_string('''
units lj
atom_style atomic
lattice fcc 0.8442
region box block 0 4 0 4 0 4
create_box 1 box
create_atoms 1 box
mass 1 1.0
pair_style lj/cut 2.5
pair_coeff 1 1 1.0 1.0 2.5
dump 1 all netcdf 5 test.nc id type x y z
run 10
''')
print('netcdf ok')
"
echo "exit: $?"
```

Passed, exit code 0.

10.4 Embedded Python package (PKG_PYTHON)

```
python -c "
from lammps import lammps
l = lammps()
l.commands_string('''
python test_fn input 0 return v_result format f here "\"\""
def test_fn():
    return 1.0
\"\"\"
''')
print('python-embed ok')
"
echo "exit: $?"
```

Passed, exit code 0.

10.5 KIM API presence

```
python -c "  
from lammmps import lammmps  
l = lammmps()  
print('KIM present:', 'KIM' in l.installed_packages)  
"
```

Confirmed True. Only the API/library was verified here; no actual KIM model data was installed or tested — model data is a separate step (Section ??).

11 Complete command sequence, start to finish

```
-- 1. Start container (from LAMMPS source root on host) -----  
docker run -it --rm -v "$PWD":/work -w /work \  
    quay.io/pypa/manylinux_2_28_x86_64:latest  
  
cd /work  
  
# -- 2. Install dnf packages inside container -----  
dnf install -y \  
    openmpi openmpi-devel \  
    fftw-devel \  
    libjpeg-turbo-devel \  
    libpng-devel \  
    zlib-devel \  
    libcurl-devel \  
    netcdf-devel \  
    openblas-devel \  
    lapack-devel \  
    ncurses-devel \  
    git  
  
# Confirm mpicc/mpicxx on PATH  
which mpicxx || export PATH=/usr/lib64/openmpi/bin:$PATH  
  
# EPEL for voro++-devel (optional; DOWNLOAD_VORO=yes is used instead here)  
dnf install -y epel-release  
  
# ffmpeg from RPM Fusion  
dnf install -y \  
    https://download1.rpmfusion.org/free/el/rpmfusion-free-release-8.noarch.rpm  
dnf install -y ffmpeg  
  
# -- 3. uv + embedding-capable Python -----  
curl -LsSf https://astral.sh/uv/install.sh | sh  
source $HOME/.local/bin/env  
uv python install 3.12  
UVPY=$(uv python find 3.12)
```

```

export PYBIN=/opt/python/cp312-cp312/bin

# Cython under uv Python (for ML-IAP mliappy)
uv pip install --python $UVPY cython

# -- 4. Configure -----
rm -rf build && mkdir build && cd build
cmake -B . -S ../cmake \
  -D BUILD_SHARED_LIBS=yes \
  -D CMAKE_BUILD_TYPE=Release \
  -D PKG_MLIAP=yes \
  -D PKG_EXTRA-PAIR=yes \
  -D PKG_KIM=yes \
  -D PKG_MDI=yes \
  -D PKG_PYTHON=yes \
  -D PKG_COLVARS=yes \
  -D PKG_NETCDF=yes \
  -D PKG_VORONOI=yes \
  -D PKG_ML-HDNNP=yes \
  -D PKG_PLUMED=yes \
  -D PKG_ML-QUIP=yes \
  -D PKG_ADIOS=no \
  -D MLIAP_ENABLE_PYTHON=no \
  -D BUILD_MPI=yes \
  -D DOWNLOAD_KIM=yes \
  -D DOWNLOAD_MDI=yes \
  -D DOWNLOAD_N2P2=yes \
  -D DOWNLOAD_VORO=yes \
  -D DOWNLOAD_PLUMED=yes \
  -D DOWNLOAD_QUIP=yes \
  -D DOWNLOAD_EIGEN3=yes \
  -D WITH_FFMPEG=yes \
  -D Python_EXECUTABLE=$UVPY

# -- 5. Build -----
make -j$(nproc)

# -- 6. Package the wheel -----
cd /work/python

$PYBIN/python install.py \
  -p lammps \
  -l ../build/liblammps.so \
  -v ../src/version.h \
  -n \
  -w .

# -- 7. auditwheel repair -----
$PYBIN/pip install auditwheel patchelf

mkdir -p wheelhouse
$PYBIN/python -m auditwheel repair \
  lammps-*-cp312-cp312-linux_x86_64.whl \

```

```
-w wheelhouse/ \  
--exclude libmpi.so.40 \  
--exclude libmpi_cxx.so.40 \  
--exclude libldap-2.5.so.0 \  
--exclude libcurl.so.4  
  
ls wheelhouse/  
# Expected: lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64  
  .whl  
  
# -- 8. Install on target (run on target machine, not in container) -----  
#   uv pip install lammps-*-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl  
#   pip install lammps-*-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
```

Part II

Part B — Native apt build (Ubuntu / Pop!_OS)

Scope

Package list for building LAMMPS natively on Ubuntu/Pop!_OS (no Docker, no manylinux container). This produces a *non-portable* wheel — it links against this machine's glibc and does not have a manylinux tag. Fine on this same OS; risky elsewhere (see Section 21).

This build was confirmed working on Pop!_OS with Python 3.12. Only LAMMPS itself is built from source. Everything it depends on comes from apt.

12 Base build tooling

```
sudo apt update
sudo apt install -y --no-install-recommends \
    ca-certificates gnupg wget curl git \
    software-properties-common apt-transport-https lsb-release \
    build-essential ninja-build pkg-config
```

13 CMake + ccmake (Kitware repo)

The distro-shipped cmake is typically too old. Use the Kitware repo:

```
wget -O - https://apt.kitware.com/keys/kitware-archive-latest.asc 2>/dev/null \
| gpg --dearmor - \
| sudo tee /usr/share/keyrings/kitware-archive-keyring.gpg >/dev/null

echo "deb [signed-by=/usr/share/keyrings/kitware-archive-keyring.gpg] \
https://apt.kitware.com/ubuntu/ $(lsb_release -cs) main" \
| sudo tee /etc/apt/sources.list.d/kitware.list

sudo apt update
sudo apt install -y cmake cmake-curses-gui

cmake --version && ccmake --version
```

cmake-curses-gui is the package that ships ccmake. The pip cmake wheel is built without curses and will not give you it.

14 Compilers and MPI (OpenMPI)

```
sudo apt install -y \  
    gcc g++ gfortran \  
    libopenmpi-dev openmpi-bin openmpi-common
```

If cmake reports Could NOT find MPI (missing: MPI_MPICXX_FOUND MPICXX) despite MPI being installed, the issue is almost always libopenmpi-dev/openmpi-bin present but the mpicxx wrapper compiler missing — the line above covers it.

Any target machine you later copy this wheel to needs an ABI-compatible OpenMPI runtime installed, or `import lammps` fails at load time on an unresolved `libmpi.so` symbol.

15 Python and wheel tooling

```
sudo apt install -y \  
    python3 python3-dev python3-pip python3-venv python3-full  
  
python3 -m pip install --break-system-packages \  
    --upgrade pip build wheel auditwheel patchelf
```

python3.12-dev note

`python3-dev` normally pulls in headers for whatever `python3` defaults to. If cmake or the embed step can't find `Python.h` specifically for 3.12:

```
sudo apt install python3.12-dev
```

This is the package that ships `Python.h` and `libpython3.12.so` headers/import libs when the generic `python3-dev` isn't resolving to 3.12.

16 All libraries (MLIAP, EXTRA-PAIR, PYTHON, COLVARS, FFT, dump stack)

```
sudo apt install -y \  
    libfftw3-dev \  
    libeigen3-dev \  
    libcurl4-openssl-dev \  
    zlib1g-dev \  
    libjpeg-dev \  
    libpng-dev \  
    libzstd-dev
```

```
libnetcdf-dev \  
libpnetcdf-dev \  
libgsl-dev \  
libopenblas-dev \  
libhdf5-serial-dev \  
libkim-api-dev \  
cython3 \  
ffmpeg
```

Notes:

- `cython3` — ML-IAP's Python model path (`mliappy`) needs a working `cythonize`.
- `libnetcdf-dev` / `libpnetcdf-dev` — matches `LMP_HAS_NETCDF`; `pnetcdf` resolves the `Could NOT find PNetCDF` cmake error.
- `libgsl-dev`, `libopenblas-dev`, `libhdf5-serial-dev` — confirmed present in the working build's `ldd liblammps.so` output (`COLVARS/LEPTON` pulls `GSL`; `BLAS` and `HDF5` come in via the `netCDF`/other package chain).
- `libkim-api-dev` — system KIM. Use this with `-D DOWNLOAD_KIM=no` in cmake to install KIM as a normal system library rather than building it in-tree. This is what makes `libkim-api.so.2` resolve to `/usr/lib/x86_64-linux-gnu/libkim-api.so.2` instead of a build-tree path that disappears when the build directory is deleted.
- `ffmpeg` — runtime binary LAMMPS pipes movie dumps to; also needed at cmake-configure time for detection.

17 cmake configure for native build

```
rm -rf build && mkdir build && cd build

cmake -B . -S ../cmake \
  -D BUILD_SHARED_LIBS=yes \
  -D CMAKE_BUILD_TYPE=Release \
  -D PKG_MLIAP=yes \
  -D PKG_EXTRA-PAIR=yes \
  -D PKG_KIM=yes \
  -D PKG_MDI=yes \
  -D PKG_PYTHON=yes \
  -D PKG_COLVARS=yes \
  -D PKG_NETCDF=yes \
  -D PKG_VORONOI=yes \
  -D MLIAP_ENABLE_PYTHON=no \
  -D BUILD_MPI=yes \
  -D DOWNLOAD_KIM=no \
  -D DOWNLOAD_MDI=yes \
  -D Python_EXECUTABLE=$(which python3.12)

make -j$(nproc)
```

Key differences from the manylinux build:

- `DOWNLOAD_KIM=no` — use `libkim-api-dev` from apt; do not build an in-tree copy that would become unresolvable the moment `build/` is deleted.
- `Python_EXECUTABLE=$(which python3.12)` — use the system Python; no uv setup needed. The system Python has a shared libpython.
- `DOWNLOAD_VORO=yes` omitted — can use system `libvoro++-dev` (`apt install libvoro++-dev`), or add `-D PKG_VORONOI=yes -D DOWNLOAD_VORO=yes` for the in-tree build.

18 Post-build sanity checks

1. Confirm a shared lib was produced

```
grep BUILD_SHARED_LIBS build/CMakeCache.txt
ls -la build/liblammps*
```

Expected: `BUILD_SHARED_LIBS:BOOL=YES` (or `ON`) and a `liblammps.so` file present — not just `liblammps.a`. If it's `OFF` or only the `.a` exists, reconfigure with `-D BUILD_SHARED_LIBS=yes` and rebuild; nothing past this point works off a static-only build.

2. Confirm the binary runs

```
build/lmp -h
```

Expected: help/usage text with package list.

3. Check the shared-lib dependency chain

```
ldd build/liblammps.so
```

Expected: every entry resolves to a real path, no "not found" lines. With `libkim-api-dev` installed and `DOWNLOAD_KIM=no` set, `libkim-api.so.2` should resolve to `/usr/lib/x86_64-linux-gnu/libkim-api.so.2` — a normal system path. If it still points into your build tree, `cmake` picked up the in-tree copy anyway; reconfigure with `DOWNLOAD_KIM=no` and confirm `KIM-API_FOUND` in `cmake` output.

19 Build the wheel (native)

```
cd build
make install-python
```

Or, to get just the `.whl` without installing into the current env:

```
python3 ../python/install.py \
-n \
-p ../python/lammps \
-l liblammps.so \
-v ../src/version.h \
-w .
```

20 Runtime smoke test (native build)

```
python3 -c "import lammps; l = lammps.lammps(); print('ok')"
```

Expected output (confirmed):

```
LAMMPS (11 Feb 2026)
  using 1 OpenMP thread(s) per MPI task
ok
```

Total wall time: 0:00:00

21 Portability caveats for the native-built wheel

This wheel is *not* `auditwheel`-repaired and dynamically links system libraries as-is:

- **glibc floor** — carries whatever glibc this Pop!_OS toolchain compiled against. An older-glibc target (e.g. older RHEL/CentOS) can fail to import.
- **OpenMPI** — target needs an ABI-compatible OpenMPI runtime.
- **KIM, now resolved** — with `libkim-api-dev` installed and `DOWNLOAD_KIM=no`, KIM resolves as a normal system library. The original landmine (unresolvable `libkim-api.so.2` path pointing into a deleted build tree) only applied to `DOWNLOAD_KIM=yes`.
- **Python-version lock** — this wheel is compiled against one specific Python minor version (3.12). It will not import under 3.11, 3.13, or any other. Every machine this wheel goes to needs a 3.12 interpreter.

For a portable wheel: use Part A (manylinux Docker build) instead.

Part III

Part C — Target machine runtime requirements

Scope

These packages are *not* bundled in the manylinux wheel and must be present on any target machine before `pip install / uv pip install` of the wheel, or `import lammps` will fail or crash at exit exactly as it did on Pop!_OS before the exclude fix.

Python: must be 3.12 specifically — the wheel is tagged `cp312`, will not import under 3.11/3.13/anything else. Confirm before installing:

```
python3 --version    # must print 3.12.x
```

22 Debian / Ubuntu / Pop!_OS target

```
sudo apt update
sudo apt install -y \
    libopenmpi3 openmpi-bin \
    libcurl4 \
    libldap-2.5-0 \
    ffmpeg
```

Notes:

- `libopenmpi3` — runtime MPI library, matches the v40-series ABI the wheel links against.
- `openmpi-bin` — provides `mpirun`, needed for multi-rank use.
- `libcurl4 / libldap-2.5-0` — excluded from vendoring to prevent the shutdown-time segfault; must be present on the system. Both were already present by default on Pop!_OS and confirmed sufficient.
- `ffmpeg` — optional, only needed for `dump movie`.

This is the set that was **verified working** on the actual Pop!_OS target machine in this build.

23 RHEL / AlmaLinux / Rocky Linux target

```
sudo dnf install -y \
    openmpi \
    libcurl \
    openldap
```

```
# ffmpeg from RPM Fusion (not in base/EPEL on RHEL-family):
sudo dnf install -y \
    https://download1.rpmfusion.org/free/el/rpmfusion-free-release-8.noarch.rpm
sudo dnf install -y ffmpeg
```

Note: not verified end-to-end on an actual RHEL-family target in this session (only Pop!_OS was tested). Package names are correct for the family but treat this as unverified relative to the Debian section above, which was directly confirmed.

24 Install the wheel

```
Any of these are equivalent once runtime deps above are installed:
uv pip install lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.
    manylinux_2_28_x86_64.whl
pip install lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.
    manylinux_2_28_x86_64.whl
```

25 KIM model data (either target)

The wheel ships the KIM API library only, not model data. If KIM potentials are used at runtime:

```
kim-api-collections-management install user <model-name>
```

26 Quick functional verification on target

```
Basic import and package list
python -c "from lammps import lammps; l = lammps(); print(l.installed_packages)"
echo "exit: $?"

# MPI
mpirun -np 2 python -c \
    "from lammps import lammps; l = lammps(); \
    l.commands_string('units lj\natom_style atomic\nlattice fcc 0.8442\n\
region box block 0 4 0 4 0 4\ncreate_box 1 box\ncreate_atoms 1 box\n\
mass 1 1.0\npair_style lj/cut 2.5\npair_coeff 1 1 1.0 1.0 2.5\nrun 10'); \
    print('mpi ok')" -quiet
echo "mpi exit: $?"
```

27 Recommended: pin this wheel via a uv project

The Python-version lock (3.12 only) has a practical consequence when running multiple MLIP backends (MACE, SevenNet, ORB, FAIRChem, PET-MAD) against this LAMMPS build. The clean way to manage this is a single uv project with the wheel referenced by local path in `[tool.uv.sources]`, `requires-python` pinned to 3.12, and each MLIP backend declared as a mutually-exclusive optional extra:

```
[project]
name = "mlip"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12,<3.13"
dependencies = [
    "ase>=3.27.0",
    "cmake>=4.2.3",
    "cryspy>=0.10.0",
    "cupy-cuda12x>=14.0.1",
    "dill>=0.4.1",
    "dpmtools>=1.1.2",
    "elastic>=5.2.5.3",
    "fitz>=0.0.1.dev2",
    "frontend>=0.0.3",
    "gpu4pyscf-cuda12x>=1.7.0",
    "imageio[ffmpeg]>=2.37.3",
    "lammps",
    "ligpargen>=2.1",
    "mdanalysis>=2.10.0",
    "mendeleeev>=1.1.0",
    "moltemplate>=2.22.5",
    "networkx>=3.6.1",
    "notebook>=7.5.5",
    "opencv-python>=4.13.0.92",
    "openmm[cuda12]>=8.5.1",
    "phono3py>=3.28.0",
    "phonopy>=3.1.0",
    "pymatgen>=2025.10.7",
    "pymupdf>=1.27.2.2",
    "pyqt6>=6.10.2",
    "python-pptx>=1.0.2",
    "rdkit>=2025.9.6",
    "skala>=2026.1",
    "spglib>=2.7.0",
    "statsmodels>=0.14.6",
]
[project.optional-dependencies]
mace = [
    "mace-torch[cpu,cuda-12]",
    "cuequivariance>=0.11.0",
    "cuequivariance-torch>=0.11.0",
    "cuequivariance-ops-torch-cu12>=0.11.0",
]
```

```

]
sevensnet = ["sevensnet[cpu]>=0.13.0"]
orb = ["orb-models>=0.5.5"]
fairchem = ["fairchem-core>=2.20.0"]
petmad = ["pet-mad"]
[tool.uv]
conflicts = [
[
{ extra = "mace" },
{ extra = "sevensnet" },
{ extra = "orb" },
{ extra = "fairchem" },
{ extra = "petmad" },
],
]

[tool.uv.sources]
lammps = { url = "https://github.com/sampk1203/lammps-mlip-wheel/releases/download/
v2026.2.11/lammps-2026.2.11-cp312-cp312-manylinux_2_27_x86_64.
manylinux_2_28_x86_64.whl" }

```

Why this matters:

- `requires-python = ">=3.12,<3.13"` enforces the 3.12-only constraint at project-resolution level — uv refuses to solve against any other interpreter.
- `tool.uv.sources.lammps` points at the `.whl` directly by path instead of pulling from PyPI — lammps becomes a normal declared dependency, not a manual `pip install /path/to/wheel.whl` step to repeat in every new env.
- `tool.uv.conflicts` stops uv from trying to resolve two MLIP backends into the same lock simultaneously. Switch backends with `uv sync -extra mace / uv sync -extra orb` etc.

A Unrelated: A100 CUDA export fix

From a separate GPU/MLIP debugging session on an A100 machine (SevenNet's D3 dispersion-correction JIT-compile a CUDA extension at runtime). Not part of the CPU wheel build.

Symptom: `nvcc fatal: Unsupported gpu architecture 'compute_61'` — CUDA 13.0 `nvcc` refuses old Pascal-era `sm_61` in the default wide arch list, even though the actual GPU (A100 = `sm_80`) is fine.

Fix, on the A100 box, before running:

```
export TORCH_CUDA_ARCH_LIST="8.0"
```

Scopes the build to just the A100's compute capability, so `nvcc` never touches the unsupported old arch. Session-only — add to `venv activate script` or `.bashrc` if persistence is needed.